

COMP-3150 CHAPTER 14: LECTURE NOTES (Ch 14)

Chapter 14: Basics of Functional Dependencies and Normalization for Relational Databases

*** Note that Midterm 1 is written in the class of Thursday, Oct 2, 2025 in class. Come to class as usual and write the test. The following are the instructions to follow for Midterm 1 so you do not waste too much time going over them during the test:

INSTRUCTIONS (Please Read Carefully)

Examination Period is 1 hours 20 minutes

Answer all questions. Write your answers in the spaces provided in the question paper. This is closed book and closed notes test.

Total Marks =50. Total number of sections = 2

Please read questions carefully! Misinterpreting a question intentionally or unintentionally results in getting a “ZERO” for that question. Good Luck!!!

CONFIDENTIALITY AGREEMENT & STATEMENT OF HONESTY

I confirm that I will keep the content of this assignment/examination confidential.

I confirm that I have not received any unauthorized assistance in preparing for or doing examination. I confirm knowing that a mark of 0 may be assigned for copied work.

For Online Test/Examination in Comp 3150: (additional rules to be observed):

- 1. I confirm that I agree to write this final examination as a closed book examination.**
- 2. I confirm that I am the student with the name and student id signed below.**
- 3. I confirm that I agree to not send email, chat, text or talk in any way to people other than the instructor or proctoring GA of this course during this examination.**
- 4. I confirm that I agree to not engage in copying or cheating during this online examination.**

Student Signature

Student Name (please print)

Student I.D. Number

Date

NOTE THAT I WILL BE GOING OVER IN THE LAST 10 MINUTES OF A CLASS YOUR ANSWERS TO THE MIDTERM 1 PRACTICE TEST I POSTED FOR YOU. NO FORMAL SOLUTIONS TO ANY PRACTICE TEST IS POSTED ON THE WEB. HOWEVER, IT WILL BE REVIEWED WITH YOU IN THIS CLASS. PARTS OF THE SOLUTION MAY ALSO BE FOUND AT THE END OF THIS LECTURE NOTES.

The following questions on **Basics of Functional Dependencies and Normalization for Relational Databases** discussed in Chapter 14 of Comp 3150 text book, Chapter 14, Comp 3150 posted course slide notes, are in-class questions for students to ponder and answer as I teach.

- The answers to the questions are found also by reviewing the Comp 3150, posted power point slide notes for Chapter 14 and being in class.
- Students are advised to review Chapters 14 of course book and Comp 3150 posted slide notes before and after each class.
- I will also go over the Slide notes in class with examples and integrate them into the class lectures, which are also posted in the More course material link on black board with any links to recorded live lectures.

1. HOW DO WE KNOW THAT OUR DATABASE DESIGN IS GOOD AND WOULD WORK WELL?

- The course book specifies 4 informal design guidelines used to measure the quality of relation schema design.

i. Attributes in the schema have clear semantics.

Guideline 1: Design a relation with clear meaning and which does not combine attributes from multiple entity types such as Employee and Department. (See an example of this combination below and in Ch. 14 slides).

**** The problem is: Storing relations that combine attributes from more than one entity type would lead to anomalies (which are insertion, deletion and modification (update) anomalies). Anomalies indicate presence of redundant storage of information in the DB design.**

ii. Redundant information in tuples are reduced.

GUIDELINE 2: Design a schema that does not suffer from the insertion, deletion and update anomalies. (See examples of what these anomalies are, below and the Ch. 14 slides).

**** Problem is: presence of these anomalies in a relation creates redundancies in the database that waste storage and make data sometimes incorrect and hard to update (e.g. Figs. 14.3 and 14.4 on Slides 12 and 13 obtained from a type of natural join of relations in Fig. 14.1 and 14.2).**

iii. NULL values in tuples are reduced.

GUIDELINE 3: Relations should be designed such that their tuples will have as few NULL values as possible. Attributes that are NULL frequently could be placed in separate relations (with the primary key).

- For example, in a University DB, rather than have an entity type called Person (SSN, Name, officePhone), where only about 6% of the 15,000 tuples are staff and faculty who have office phone numbers and the rest are students who do not and have NULL values, it is better to design separate entity types for the same information such as:

STUDENT(SID, Name),

STAFF(STID, SNAME, OFFICEPHONE).

iv. Possibility of generating spurious tuples are disallowed.

GUIDELINE 4: The relations should be designed to satisfy the lossless join condition.

- No spurious (fake) tuples (not in the original relation) should be generated by doing a natural-join of any decomposed relations.

- Design relation schemas that can be joined with equality

conditions on attributes that are (primary key, foreign key) pairs.

- There are two important properties of decompositions of relations to keep the design good:
 - a) Non-additive or losslessness of the corresponding join (that is, not introducing new attributes or tuples and not losing existing attributes or tuples when decomposed relations are joined back).
 - b) Preservation of the functional dependencies (FDs) (that is, the FDs holding in the bad relations in the whole DB before decomposition, should be holding in the whole DB after decomposition but each relation should be free of anomalous or violating FDs).
- An example of decomposition of a relation R that yields spurious tuples is given below.
- Let R called Properties(Ssn, books, pens) be the original relation and let this relation be decomposed into two relations Properties1(Ssn, books) and Properties2(books, pens)

(a) Original relation

Two decomposed relations (b) and (c)

Properties			Properties1		Properties2	
Ssn	Books	Pens	Ssn	Books	Books	Pens
1	12	8	1	12	12	8
2	25	5	2	25	25	5

Result of joining back decomposed relations Properties1 and Properties2

Properties1 join Properties2

Ssn	Books	Pens
1	12	8
1	25	5
2	12	8
2	25	5

** The spurious tuples not in the original relation
Properties are the 2 highlighted middle tuples.

- A correct decomposition of Properties is shown below:

(a) Original relation

Two decomposed relations (b) and (c)

Properties			Properties3		Properties4	
Ssn	Books	Pens	Ssn	Books	Ssn	Pens
1	12	8	1	12	1	8
2	25	5	2	25	2	5

Result of joining back decomposed relations Properties3 and Properties4

Properties3 join Properties4

Ssn	Books	Pens	
1	12	8	
2	25	5	** No spurious tuples not in the original relation
Properties are generated here.			

Note that:

Property (a) is extremely important and cannot be sacrificed. Property (b) is less stringent and may be sacrificed.

2. WHAT ARE FUNCTIONAL DEPENDENCIES?

3. HOW DO WE DECOMPOSE RELATIONS INTO NORMALISED RELATIONS THAT ARE IN THE 1ST NORMAL FORM, 2ND NORMAL FORM, 3RD NORMAL FORM, BOYCE CODD NORMAL FORM (BCNF)?

(ALSO, 4TH AND 5TH NORMAL FORM EXIST But not addressed in this course).

- Note that a well-designed normalized relation needs to be in at least the 3rd normal form.

A fast example on normal forms:

Given the relation

Takes (Sid, Cid, Score, Grade)

Is this relation in 3NF?

- A relation is in 3NF if only the whole of the primary key uniquely determine all other attributes and no partial or transitive dependencies exist.

The functional dependencies that exist in this Takes are:

FD1: (Sid, Cid) $\rightarrow$ {Score, Grade} // because it is the PK

FD2: Score $\rightarrow$ Grade // this is transitive dependency

- Takes is NOT in 3NF because of FD2 that violates the 3NF rule.
- Thus, Takes contains anomalies.
- Can we decompose Takes to keep the database in 3NF?

- To put the DB in 3NF, create new tables with each violating FD. And remove all the right hand attributes in the violating FDs from the original DB table, which is also kept as a table in the DB.
- The normalized database form of Takes above are:

Takes1 (Sid, Cid, Score)

Score1(Score, Grade)

The functional dependencies that exist in this new normalized DB are:

FD1: (Sid, Cid) $\rightarrow$ {Score} // because it is the PK
// Takes1

FD2: Score $\rightarrow$ Grade // for PK in Score1

- Thus, although the FDs in the 3NF DB are the same as those in the non-normalized table Takes, those FDs are in different normalized tables in Takes1 and Score1.

Examples Describing Guideline 1: Combination of Entity types in one relation.

Slide 6 of Course Slide Notes of Chapter 14 shows Fig. 14.1, a simplified COMPANY relational DB schema. A state of this DB is Fig. 14.2.

Figure 14.1 A simplified COMPANY relational database schema.

EMPLOYEE					F.K.
Ename	<u>Ssn</u>	Bdate	Address	Dnumber	
P.K.					

DEPARTMENT			F.K.
Dname	<u>Dnumber</u>	Dmgr_ssn	
P.K.			

DEPT_LOCATIONS		F.K.
<u>Dnumber</u>	<u>Dlocation</u>	
P.K.		

PROJECT				F.K.
Pname	<u>Pnumber</u>	Plocation	Dnum	
P.K.				

WORKS_ON			F.K.	F.K.
<u>Ssn</u>	<u>Pnumber</u>	Hours		
P.K.				

Figure 14.2 Sample database state for the relational database schema in Figure 14.1.

EMPLOYEE

Ename	<u>Ssn</u>	Bdate	Address	Dnumber
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4
Wallace, Jennifer S.	987654321	1941-06-20	291 Berry, Bellaire, TX	4
Narayan, Ramesh K.	666884444	1962-09-15	975 Fire Oak, Humble, TX	5
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX	5
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX	4
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX	1

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn
Research	5	333445555
Administration	4	987654321
Headquarters	1	888665555

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

WORKS_ON

<u>Ssn</u>	<u>Pnumber</u>	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	Null

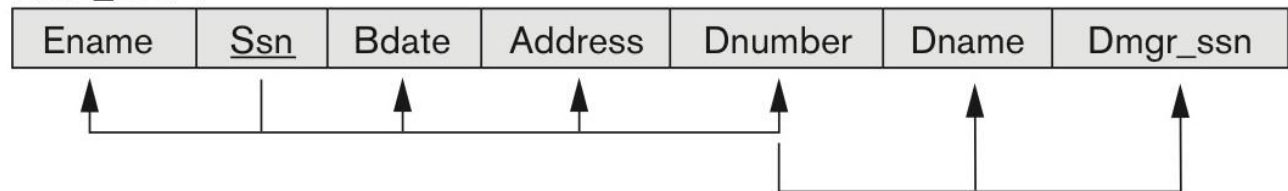
PROJECT

Pname	<u>Pnumber</u>	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

Figure 14.3 Two relation schemas suffering from update anomalies. (a) EMP_DEPT and (b) EMP_PROJ.

(a)

EMP_DEPT



(b)

EMP_PROJ

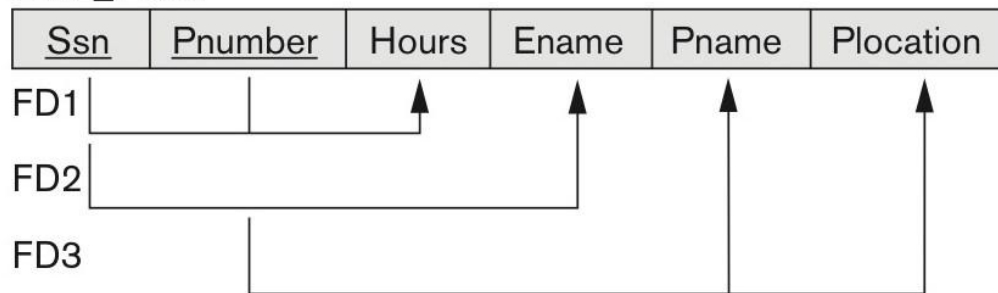


Figure 14.4 Sample states for EMP_DEPT and EMP_PROJ resulting from applying NATURAL JOIN to the relations in Figure 14.2. These may be stored as base relations for performance reasons.

					Redundancy	
EMP_DEPT						
Ename	<u>Ssn</u>	Bdate	Address	Dnumber	Dname	Dmgr_ssn
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5	Research	333445555
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5	Research	333445555
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4	Administration	987654321
Wallace, Jennifer S.	987654321	1941-06-20	291 Berry, Bellaire, TX	4	Administration	987654321
Narayan, Ramesh K.	666884444	1962-09-15	975 FireOak, Humble, TX	5	Research	333445555
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX	5	Research	333445555
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX	4	Administration	987654321
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX	1	Headquarters	888665555

			Redundancy		Redundancy	
EMP_PROJ						
Ssn	Pnumber	Hours	Ename	Pname	Plocation	
123456789	1	32.5	Smith, John B.	ProductX	Bellaire	
123456789	2	7.5	Smith, John B.	ProductY	Sugarland	
666884444	3	40.0	Narayan, Ramesh K.	ProductZ	Houston	
453453453	1	20.0	English, Joyce A.	ProductX	Bellaire	
453453453	2	20.0	English, Joyce A.	ProductY	Sugarland	
333445555	2	10.0	Wong, Franklin T.	ProductY	Sugarland	
333445555	3	10.0	Wong, Franklin T.	ProductZ	Houston	
333445555	10	10.0	Wong, Franklin T.	Computerization	Stafford	
333445555	20	10.0	Wong, Franklin T.	Reorganization	Houston	
999887777	30	30.0	Zelaya, Alicia J.	Newbenefits	Stafford	
999887777	10	10.0	Zelaya, Alicia J.	Computerization	Stafford	
987987987	10	35.0	Jabbar, Ahmad V.	Computerization	Stafford	
987987987	30	5.0	Jabbar, Ahmad V.	Newbenefits	Stafford	
987654321	30	20.0	Wallace, Jennifer S.	Newbenefits	Stafford	
987654321	20	15.0	Wallace, Jennifer S.	Reorganization	Houston	
888665555	20	Null	Borg, James E.	Reorganization	Houston	

```
SQL> select ssn, pno, fname, lname, pname, hours
2  from employee, works_on, project
3  where ssn = essn and pno = pnumber;
```

SSN	PNO	FNAME	LNAME	PNAME	HOURS
999887777	10	Alicia	Zelaya	Computerize	10
987987987	10	Ahmad	Jabbar	Computerize	35
333445555	10	Franklin	Wong	Computerize	10
999887777	30	Alicia	Zelaya	Nbenefits	30
987987987	30	Ahmad	Jabbar	Nbenefits	5
987654321	30	Jennifer	Wallace	Nbenefits	20
453453453	1	Joyce	English	ProductX	20
123456789	1	John	Smith	ProductX	32.5
453453453	2	Joyce	English	ProductY	20
333445555	2	Franklin	Wong	ProductY	10
123456789	2	John	Smith	ProductY	7.5

SSN	PNO	FNAME	LNAME	PNAME	HOURS
-----	-----	-------	-------	-------	-------

666884444	3	Ramesh	Narayan	ProductZ	40
333445555	3	Franklin	Wong	ProductZ	10
987654321	20	Jennifer	Wallace	Reorganize	15
888665555	20	James	Borg	Reorganize	10
333445555	20	Franklin	Wong	Reorganize	10

16 rows selected.

What are the Meanings of the Anomalies in Guideline 2?
Explained in Slides 8 to 15.

Consider the relation:

EMP_PROJ(Emp#, Proj#, Ename, Pname, No_hours)

i. Update Anomaly: Changing the name of project number P10 from “Computerize” to “Customer-Accounting” will cause this update to be made for all 3 (may have more in a

bigger state) employees working on project P10 when information about employees working on a project do not need to be affected by changes in properties of project. The association between the Proj# and Pname must be maintained in all tuples or this integrity constraint is violated.

ii. Insert Anomaly: Cannot insert a project unless an employee is assigned to it although project number and project name do not really depend on the employees assigned to the project. Conversely, we cannot insert an employee unless he/she is assigned to a project although information about employee does not depend on the project the employee is assigned to. This is because both Emp#, Proj# are the primary key and none of these attributes of the primary key is allowed to be NULL.

iii. Delete Anomaly: When a project is deleted, it may result in deleting all the employees who work on that project causing us to lose information about these employees. Alternatively, if an employee is the sole employee on a project, deleting that employee would result in deleting the corresponding project and losing all the information about this project.

2. What are Functional Dependencies?

- Functional dependencies (FDs) are used to specify formal measures of the "goodness" of relational designs.
- FDs and keys are used to define normal forms for relations
- FDs are constraints that are derived from the meaning and inter-relationships of the data attributes.
- A set of attributes X functionally determines a set of attributes Y if the value of X determines a unique value for Y
- An example FD in a Student Information system with table TAKES (SID, CID, SCORE, GRADE) is Score $\rightarrow$ Grade. This means that in this world, you really cannot say what Grade a student has received in a course if you do not have the Score they received, interpreted as: Score determines Grade.
- However, also by their role as the unique identifier, the primary key (consisting of (SID, CID)) also determines every tuple and each attribute of the relation). Thus, the following FD also holds in the table TAKES:
 $(\underline{\text{SID}}, \underline{\text{CID}}) \rightarrow \{\text{Score}, \text{Grade}\}$

This means that (SID, CID) determines all other attributes in the table, Score and Grade.

Consider a state or instance of the table TAKES as:
TAKES

<u>SID</u>	<u>CID</u>	SCORE	GRADE
1	COMP2120	80	A-
2	COMP2120	85	A
3	COMP3150	80	A-
4	COMP3150	85	A
5	COMP3150	77	B+

-Only the primary key should be the unique determinant of other non-key attribute or you may have insert, delete or update anomalies.

- We can see that these anomalies exist in TAKES because the FDs existing in this DB are:

FD1: (SID, CID) $\rightarrow$ {Score, Grade}

FD2: SCORE $\rightarrow$ Grade

-Note that FD2 is a violating FD in this table that is causing these anomalies and this should be handled through decomposition to put this table in at least 3NF to have it normalized.

Question:

i. Show how there is insert anomaly in this table TAKES.

-Although it is not the primary key, we cannot insert a student record taking this course with a NULL score in this table. We must wait until they receive a score in the course to indicate the course they have taken. This is not proper as this table should be able to record all students and courses, and grades they received in the course, even if they did not complete this course and have no grade. Thus, we cannot insert a tuple like $\langle 6, \text{'COMP2120'}, \text{NULL}, \text{NULL} \rangle$.

ii. Show how there is delete anomaly in this table TAKES.

The only way we can tell the association between Score and Grade values are by checking similar values in the table TAKES where for any two tuples (t1 and t2) of TAKES, if $t1.\text{Score} = t2.\text{Score}$, then, $t1.\text{Grade} = t2.\text{Grade}$. For example, tuples 1 and 3 have their Score = 80 and their grades = 'A-'.

Thus, for delete anomaly, if we happen to delete the last tuple with this score value by deleting both tuples 1 and 3 from the database, we no longer know the association between the score of 80 and its equivalent letter grade.

iii. Show how there is update anomaly in this table TAKES
If you have one hundred students who received a grade of 80 for example, and the grade scale recently changed from 80 to 83 to be the minimum for a grade of “A-“, we must change all 100 records from 80 to 83 or the FD that must exist in the database is violated. This is an example of update anomaly.

iv. Normalize this table TAKES to have it in 3NF and remove these anomalies so you create a normalized database where all of its tables are normalized and the FDs in this TAKES will be preserved in the new normalized DB with no loss of tuples/attributes if joined back.

- The decomposition is better done using the FDs and the rules about 1NF, 2NF and 3NF as discussed on slides 24 to 45 of Ch. 14 slide notes.
- An informal summary of 1NF, 2NF and 3NF rules as on slide 39 is given below:

Normal Forms Defined Informally

- 1st normal form (1NF)
 - All attributes depend on the key and are single valued.
- 2nd normal form (2NF)
 - All attributes depend on the whole key (ie, all attributes are fully functional dependent on the whole key - that is, no partial dependency).
- 3rd normal form (3NF)
 - All attributes depend on nothing but the key (i.e., no attribute should depend on the key through another attribute: no transitive dependency).

TO NORMALIZE **TAKES** RELATION

1. Identify all existing FDs in the relation including the FD imposed by the primary key, and identify the FDs that are

violating any of the 1NF, 2NF or 3NF rules.

The existing FDs in TAKES are:

FD1: (SID, CID) $\rightarrow$ {Score, Grade} ****(FD imposed by PK)**

FD2: SCORE $\rightarrow$ Grade ****(violating FD)**

2. Remove all right hand attributes of the violating FDs from the right hand side of the main PK FD, while still retaining the violating FD in the DB. This will remove all the violations from the tables and FDs. The result is:

FD1: (SID, CID) $\rightarrow$ {Score, ~~Grade~~}

FD2: SCORE $\rightarrow$ Grade.

3. Convert each of the FDs into a normalized DB table with a unique name where the primary key of the table from the FD is the left side attributes of the FD and the rest of the attributes on its right side are the other attributes as in:

TAKES (SID, CID, Score)

SCOREINFO (SCORE, Grade)

The FDs in the above normalized DB with two tables are the two FDs refined above as:

FD1: (SID, CID) $\rightarrow$ {Score}

FD2: SCORE $\rightarrow$ Grade.

Ch 14: More on Normal Forms: Slide 36 of Ch 14 Slides

Is this relation in 2NF?

Emp_Proj(Ssn, Pnumber, Hours, Ename, Pname, Plocation)

FDs:

FD1: (Ssn, Pnumber) $\rightarrow$ {Hours, Ename, Pname, Plocation}

FD2: Ssn $\rightarrow$ Ename (Violating)

FD3: Pnumber $\rightarrow$ Pname, Plocation (Violating)

2NF: A relation R is in 2NF if every non-key attribute A in R is FFD (fully functionally dependent) on the primary key (Pk) (and not on any proper subset of the PK).

** Note ~~FFD~~ (struck out) here means does not fully functionally determine.

Emp_Proj is not in 2NF because the Pk (Ssn,Pnumber) ~~FFD~~ $\rightarrow$ Ename because of FD2. In FD2 Ssn is a subset of the Pk and Ssn $\rightarrow$ Ename.

Also, the presence of FD3 shows that
(Ssn,Pnumber) ~~FFD~~ $\rightarrow$ Pname, Plocation

Normalize Emp_Proj into 2NF. Normalization is usually accomplished through decomposition of a relation to remove the violating FD's from one single table.

Emp_Proj

FD1: (Ssn, Pnumber) $\rightarrow$ {Hours, ~~Ename~~, ~~Pname~~, ~~Plocation~~}

Remove the right hand side attributes of violating FD's (FD2 and FD3).

Then, re-insert the updated FD1 with the violating FD's in the DB and derive relations from all these FD's to have your normalized DB.

FD's

FD1: (Ssn, Pnumber) $\rightarrow$ Hours

FD2: Ssn $\rightarrow$ Ename

FD3: Pnumber $\rightarrow$ {Pname, Plocation}

Create the tables from FD's where the schema consists of all attributes in each FD with left hand side attributes as the PK.

Emp_proj1 (Ssn, Pnumber, Hours)

Emp_Proj2 (Ssn, Ename)

Emp_Proj3 (Pnumber, Pname, Plocation)

Ch. 14 Continues: BCNF: Slides 41 to 44

1. An example of relation which is in 3NF but not in BCNF.

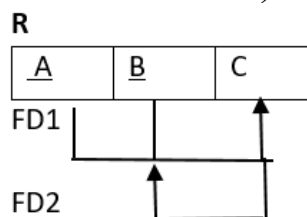
BCNF differs from the 3NF in definition because in the general definition of 3NF, as clause (b) mentioned on slide 41 allows FD's that have the right hand side of $X \rightarrow A$ as a prime (key) attribute (that is determined by a non-key attribute), is not allowed in the BCNF.

For example:

R (A, B, C), here R is a relation in 3NF but not in BCNF.

FD1: (A, B) $\rightarrow$ C

FD2: C $\rightarrow$ B (BCNF violation)



FD2 violates the BCNF, but R is in 3NF but not in BCNF.

2. How can we put R in BCNF?

In order to normalize R into BCNF, we decompose R into BCNF tables by first removing the violating FD from the R the same way we did with decomposition of non-normalized tables into 2NF and 3NF.

Normalized FDs are:

FD1: $(A, \underline{B}) \rightarrow C$ (removing right hand side of FD2 from FD1)

FD2: $C \rightarrow B$ (BCNF violation) (needs to be preserved as before)

Resulting BCNF tables arising from the FD1 and FD2 above are:

R1 (A, C)

R2 (C, B)

R1 and R2 are in BCNF. The Functional dependencies are as follows:

FD1: $A \rightarrow C$

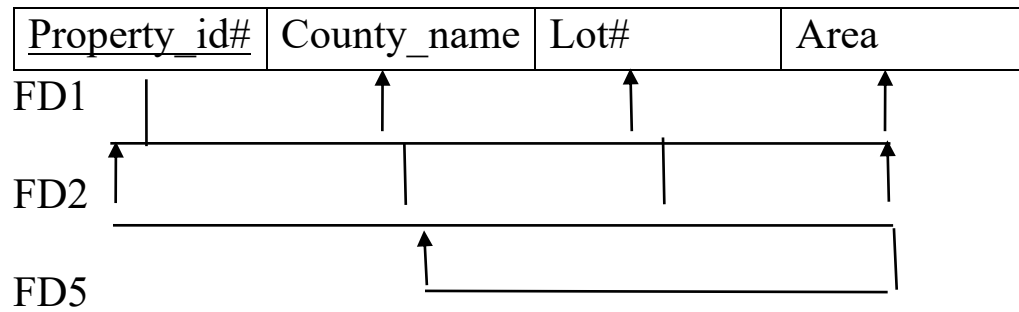
FD2: $C \rightarrow B$

—

An Example of BCNF used in the book on Figs. 14.11 to 14.13 for normalizing the LOTS1A relation into BCNF.

3. Boyce-Codd normal form. BCNF normalization of LOTS1A with the functional dependency FD2 being lost in the decomposition. Note that (County_Name, Lot#) is a candidate key.

LOTS1A



FD1: Property_id# → County_name, Lot#, Area

FD2: County_name, Lot# → Property_id#, Area

FD5: Area → County_name (violates BCNF)

(County_name, Lot#) is a candidate key.

LOTS1A is in 3NF but not in BCNF. To decompose it into BCNF, remove the FD5 from the main table and re-insert it as a separate table.

After BCNF Normalization

LOTS1AX

<u>Property_id#</u>	Lot#	Area
---------------------	------	------

LOTS1AY

<u>Area</u>	County_name
-------------	-------------

LOTS1AX (Property_id#, Lot#, Area)

LOTS1AY (Area, County_name)

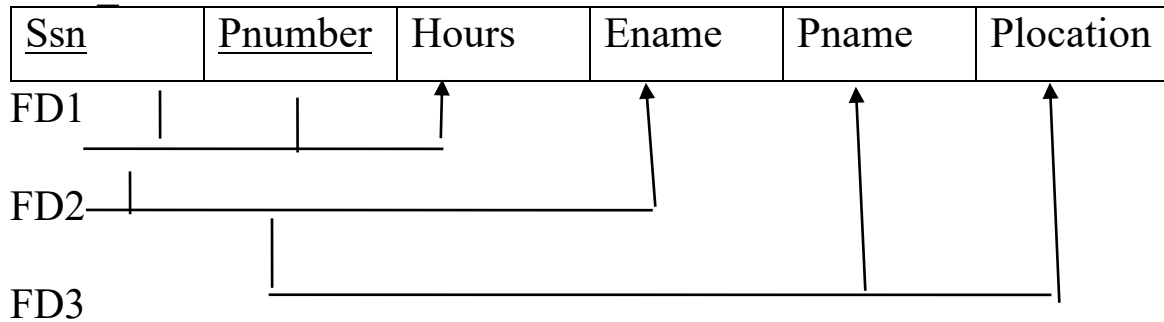
New FDs in DB are:

FD1: Property_id# → Lot#, Area

FD2: Area → County_name

4. Normalizing into 2NF and 3NF. (a) Normalizing EMP_PROJ into 2NF relations. (Done before but re-visited)

EMP_PROJ



EMP_PROJ is in 1NF but not in 2NF because FD3 is violating the 2NF condition as it requires the primary key

Ssn, Pnumber ---FFD----> Pname

FD2 is also violating as well Ssn → Ename (violating)

FD3: Pnumber → Pname, Plocation (violating)

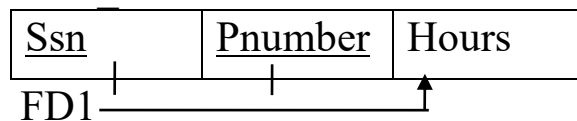
To make EMP_PROJ in 2NF, decompose it,

EMP_PROJ1 (Ssn, Pnumber, Hours, ~~Ename~~, ~~Pname~~, ~~Plocation~~)

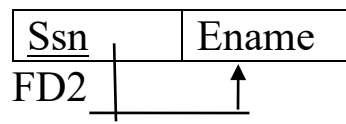
EMP_PROJ2 (Ssn, Ename)

EMP_PROJ3 (Pnumber, Pname, Plocation)

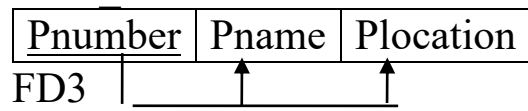
EMP_PROJ1



EMP_PROJ2



EMP_PROJ3



**** Parts of Solution to Midterm 1 Practice (handed out Fall 2023 Class)**

Section B (35 marks):

This section has 3 questions :

1. **(15 marks)** Given the ER diagram of the COMPANY database of Figure 3.2, answer the following questions with this diagram.

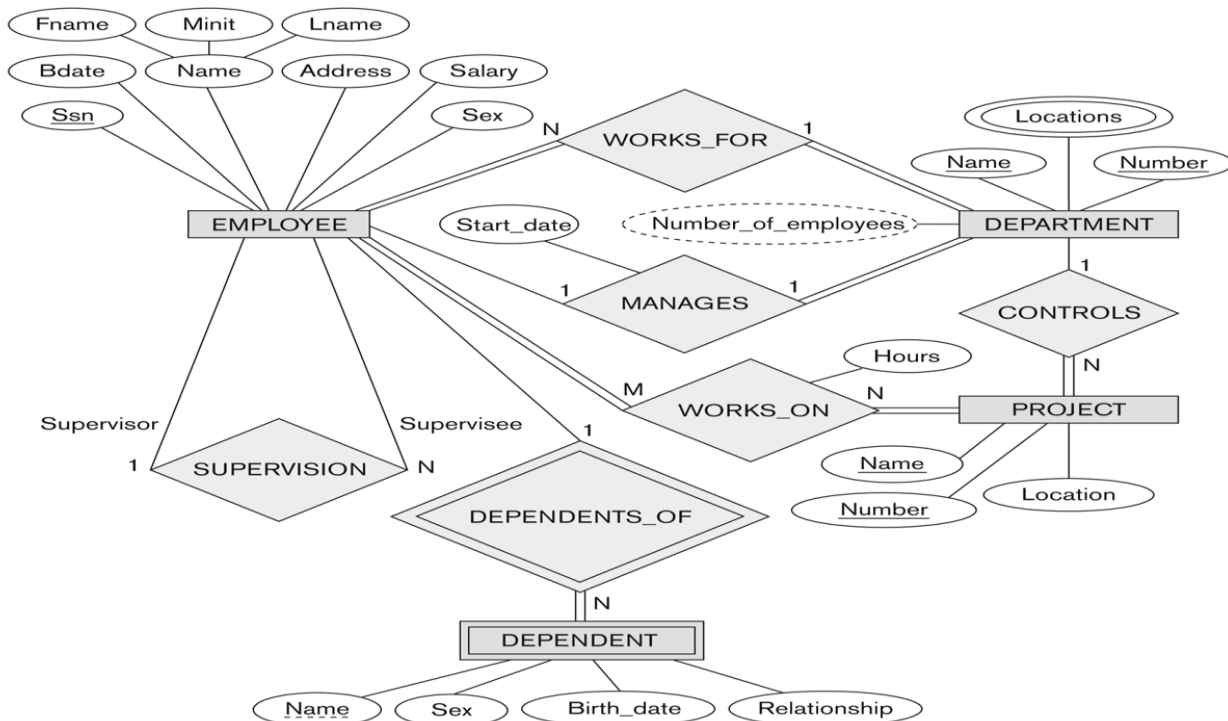


Figure 3.2

An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter.

Write the relational database schema (in text form) for COMPANY including all the entities with attributes, relationships with attributes and list at least two of the six cardinality and participation constraints that describe each of the six relationships in this database (Full description of one relationship constraints stands for one).

COMPANY Database Component	COMPANY SCHEMA for the Database Component
Entities and attributes (5 marks)	EMPLOYEE(<u>Ssn</u>, Bdate, Fname, Minit, Lname, Address, Salary, Sex) DEPARTMENT(<u>DName</u>, Location1, Location2, <u>DNumber</u>) DEPENDENT(<u>TName</u>, Sex, Birth_date, Relationship) PROJECT(<u>PName</u>, PNumber, Location) **Detailed scheme: 4 tables expected. Duck (-0.5) for each table missing. There are 4 entities tables. Take -0.5 for each incorrect attribute set. Deduct up until -2 total.

Relationships and attributes (5 marks)	SUPERVISION(<u>Sssn</u>, <u>Essn</u>) //Sssn is supervisor's and Essn is employee's WORKS_FOR(<u>Ssn</u>, <u>DNumber</u>) MANAGES(<u>SSn</u>, <u>DNumber</u>, Start_date) WORKS_ON(<u>SSn</u>, <u>PNumber</u>, Hours) CONTROLS(<u>DNumber</u>, <u>PNumber</u>) DEPENDENTS_OF(<u>Ssn</u>, <u>TName</u>) **Detailed scheme: at least 6 tables expected. Duck (-0.5) for each table missing. There are 6 relationship tables. Take -0.5 for each incorrect attribute set. Deduct up until -2 total
Cardinality and Participation Constraints (5 marks)	i. Every employee works for only one department and every department has many department working for it. ii. Every department must be managed by one employee manager and an employee manager can manage only one department. iii. Every project is controlled by only one department but a department can have many projects. iv. Every employee can work on many projects and every project can be worked on by many employees. v. An employee can have many dependents while each dependent must have an employee they are dependent-of. vi. An employee supervisor can supervise many employee supervisee while an employee supervisee can be supervised by only one employee supervisor. **Detailed scheme: at least 2 of any of the 6 descriptions expected. Duck (-2.5) for any of the 2 missing.

2. **(10 marks)** Given the following relational database schema for a phone subscription database application, answer the questions that follow.

CUSTOMER (SSN, Cname, City)
PHONE(Phonetype, PhoneMaker)
PHONECOMPANY (CompId, CompName, Numproducts)
SUBSCRIBETO (SSN, CompId, Phonetype, Month, Billamt)

Here, Numproducts refers to the number of phone products carried by the phone company. Billamt is the dollar amount of bill for the month on the phone to the customer with SSN.

- Specify all the foreign keys for this schema. State each foreign key using the following format "foreign key is the attribute ----- of relation ----- that references relation -----".
(5 marks)
- State two queries involving more than one table that can be posed on this database indicating the tables to visit to answer the queries each time. (5 marks)

Question	Foreign Key
2a (5 marks)	i. the attribute SSN of relation SUBSCRIBETO that references relation CUSTOMER, ii. the attribute <u>CompId</u> of relation SUBSCRIBETO that references relation PHONECOMPANY, iii. the attribute <u>Phonetype</u> of relation SUBSCRIBETO that references relation PHONE **Detailed scheme: Duck (-1) for each of the main foreign keys missing.

Question	Query	Tables needed to answer query
2b (5 marks)	i. Print the billamt and the month of phone-type in a Company Id by a customer (Customer name). ii. Print the city, and Company name of all customers subscribed to Company with Compld. **Detailed scheme: Duck (-1.5) for each of the English queries not given. -1 for not identifying the tables involved in answering the queries and -0.5 for each missing table up to 2.	i. CUSTOMER, SUBSCRIBETO ii. CUSTOMER, SUBSCRIBETO, PHONECOMPANY

3. (10 marks)

- Design a simple STUDENT records database with not more than 3 tables
(5 marks)
- Create an instance of your database in (a).
(5 marks)

Question	Answers																																				
a. (5 marks)	A simple student information database system with only three tables is: Enrolled(sid,<u>cid</u>, grade) Student(sid, name, email, gpa) Courses(<u>cid</u>, cname, credits) **Detailed scheme: Duck (-1.5) for each of 3 main tables not identified correctly with its attributes and primary key (PK) (-0.5 from the -1.5 if only the PK is not identified).																																				
b. (5 marks)	Student <table><tr><th>sid</th><th>name</th><th>email</th><th>gpa</th></tr><tr><td>1</td><td>John Smith</td><td>jsmith@uw.ca</td><td>90</td></tr><tr><td>2</td><td>Mary Hat</td><td>mhat@uw.ca</td><td>78</td></tr></table> Course <table><tr><th>cid</th><th>cname</th><th>credits</th></tr><tr><td>315</td><td>Databases</td><td>3</td></tr><tr><td>212</td><td>Java</td><td>3</td></tr></table> Enrolled <table><tr><th>sid</th><th>cid</th><th>grade</th></tr><tr><td>1</td><td>315</td><td>95</td></tr><tr><td>2</td><td>315</td><td>80</td></tr><tr><td>1</td><td>212</td><td>78</td></tr><tr><td>2</td><td>212</td><td>70</td></tr></table> **Detailed scheme: It is -1 for each table not filled. -1 for violating the PK and FK constraints in each table.	sid	name	email	gpa	1	John Smith	jsmith@uw.ca	90	2	Mary Hat	mhat@uw.ca	78	cid	cname	credits	315	Databases	3	212	Java	3	sid	cid	grade	1	315	95	2	315	80	1	212	78	2	212	70
sid	name	email	gpa																																		
1	John Smith	jsmith@uw.ca	90																																		
2	Mary Hat	mhat@uw.ca	78																																		
cid	cname	credits																																			
315	Databases	3																																			
212	Java	3																																			
sid	cid	grade																																			
1	315	95																																			
2	315	80																																			
1	212	78																																			
2	212	70																																			

Que	ans
1	d
2	a
3	b
4	e
5	b
6	c
7	e
8	a
9	d
10	a

The above is for section A.